MACHINE LEARNING OPERATIONS

End-to-End MLOps for Heart Disease Risk Prediction

Assignment 01 | End-to-End Model Development, CI/CD, and Deployment

AIMLCZG523

Student	Akash Ray
Student ID	2025AE05156
Repository	github.com/tupakash/MLOps-Assignment
API Base URL	https://getgd5jtgh.execute-api.ap-south-1.amazonaws.com/prod
Grafana Dashboard URL	https://g-581e8f5010.grafana-workspace.ap-southeast-1.amazonaws.com/d/heart-disease-api-cloudwatch/heart-disease-api-cloudwatch-monitoring?orgId=1&refresh=30s&from=1792363878098&to=1792385478098
Screen Recording Video Link	https://drive.google.com/file/d/1bnPeD2L0BKruUdFD4vN_IYmd4Xp_rPP0/view?usp=sharing
Report date	18 October 2026

Contents

- Abstract
- 1. Problem statement and objectives
- 2. Dataset and exploratory data analysis
- 3. Feature engineering and model development
- 4. Experiment tracking and reproducibility
- 5. Model packaging and API
- 6. CI/CD and automated testing
- 7. Deployment architecture
- 8. Monitoring and operational demonstration
- 9. Limitations and future work
- 10. Conclusion
- Appendix A. Required submission evidence

Abstract

I built an end-to-end MLOps workflow for binary heart-disease risk classification using the Cleveland Heart Disease dataset. The workflow covers data exploration, preprocessing, model comparison, MLflow tracking, ONNX packaging, a FastAPI service, Docker, CI, AWS deployment, and request-log monitoring. I compared 20 candidates from Logistic Regression, Random Forest, and XGBoost. In the recorded experiment, Logistic Regression with `C=0.1` had the best mean five-fold cross-validation ROC-AUC. On the held-out test split, it achieved ROC-AUC 0.961 and recall 0.929. I exported the fitted pipeline to ONNX and served it with ONNX Runtime.

I deployed the API with Terraform and configured AWS Managed Grafana to query its structured CloudWatch logs. This is an academic demonstration: the prediction is not medical advice or a diagnosis.

1. Problem statement and objectives

I set out to predict heart-disease presence (`target=1`) or absence (`target=0`) from 13 input features and serve the prediction through a reproducible API. The source dataset encodes the target from 0 to 4. To match the assignment's binary classification task, I mapped 0 to absence and values 1–4 to presence.

My main goals were to:

- acquire and inspect the dataset reproducibly;
- handle missing values and encode/scale features within a reusable pipeline;
- compare multiple model families using stratified validation;
- preserve experiment parameters, metrics, and artifacts in MLflow;
- export a portable inference model and expose a JSON API;
- automate linting, tests, training, image building, and deployment checks;
- collect request logs and provide metrics/log dashboards.

2. Dataset and exploratory data analysis

When the local CSV is absent, the data loader downloads the processed Cleveland dataset from the UCI Machine Learning Repository. The dataset I used contains **303 rows and 14 columns**: 13 predictors and the target. Notebook checks found no exact duplicate rows. The source represents missing `ca` and `thal` values with question marks, so I converted those markers to missing values before analysis. This revealed four missing `ca` values and two missing `thal` values.

There are 164 records with source target 0 and 139 with source targets 1–4. After binarization, the class counts are 164 (**54.1%**) and 139 (**45.9%**), respectively. The difference is modest, and I used stratified splitting to preserve the class proportions.

In `notebooks/01_eda.ipynb`, I plotted the class counts, feature histograms, target-stratified boxplots, and a correlation heatmap. I treated `ca` as a vessel count and categorical codes as categories rather than continuous measurements. In this sample, `ca` and `oldpeak` correlate positively with the binary target (about 0.46 and 0.42), while `thalach` correlates negatively (about -0.42). These small-sample associations are descriptive, not causal.

The dataset is downloaded into `src/data_loader.py`. The download checks the HTTP response before saving the file. The training pipeline records the dataset's SHA-256 in MLflow, so I can identify the exact input used for a run.

The notebook also compares distributions by target. In this sample, the disease-present group has higher medians for age, `oldpeak`, and `ca`, and a lower median maximum heart rate (`thalach`). Resting blood-pressure medians are similar, while cholesterol values overlap substantially and include outliers. I used these plots to understand the data, not to define a single-feature decision rule. I limited the Pearson heatmap to numeric/count features because correlations on category codes can be misleading. The notebook contains the plots.

2.1 Data preparation

`src.feature_engineering.prepare_features` converts the question-mark markers to missing values, validates that all fields are numeric, checks the allowed source target values, separates predictors from target, and maps the target to binary. Missing predictors are not dropped; the imputation steps are fitted within each model pipeline, preventing imputation statistics from leaking from validation/test folds into model training.

3. Feature engineering and model development

All 13 predictors are retained. The preprocessing pipeline in `src.feature_engineering.build_preprocessor` groups six numeric/count variables—`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, and `ca`—and seven categorical variables—`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, and `thal`.

Numeric variables are median-imputed and standardized. Categorical variables are imputed with the most frequent value and one-hot encoded, with unknown categories ignored at transform time. Preprocessing and the classifier are combined in a single scikit-learn pipeline so the same transformations are applied during training and inference.

I split the data into training and held-out test sets (80/20), using stratification and random state 42. On the training split I used shuffled, five-fold `StratifiedKFold` cross-validation, also with random state 42. I compared 20 candidates: four Logistic Regression `C` values, eight Random Forest configurations, and eight XGBoost configurations. I selected the candidate with the highest mean cross-validation ROC-AUC and also recorded accuracy, precision, and recall.

I implemented the search in `src.pipeline._build_candidate_pipelines`. Logistic Regression uses `max_iter=1000` and `C` values 0.01, 0.1, 1, and 10. For Random Forest I tested 100 or 200 trees, maximum depth `None` or 8, and minimum leaf size 1 or 2. For XGBoost I tested 100 or 200 estimators, maximum depth 2 or 3, and learning rate 0.03 or 0.1. Each family uses random state 42; XGBoost uses log-loss evaluation and one worker. This gives 4 + 8 + 8 = 20 candidates.

I evaluated each candidate on the same stratified folds. `cross_validate` computes accuracy, precision, recall, and ROC-AUC for each fold, and the pipeline records each metric's mean and standard deviation. I set `error_score="raise"` so a failed fit stops the run instead of receiving a placeholder score. I did not use the held-out test set for model selection.

3.1 Recorded cross-validation comparison

The recorded output in `notebooks/02_training.ipynb` gives this rounded family-level comparison:

Model family	CV accuracy	CV precision	CV recall	CV ROC-AUC
Logistic Regression	0.851	0.878	0.792	0.911
Random Forest	0.802	0.800	0.765	0.897
XGBoost	0.822	0.828	0.783	0.885

Logistic Regression with `classifier__C = 0.1` had the highest reported mean CV ROC-AUC. The automated pipeline also records fold means and standard deviations for all 20 candidates in MLflow.

3.2 Held-out test evaluation

On the recorded held-out split, the selected Logistic Regression pipeline achieved:

Metric	Held-out test result
Accuracy	0.885
Precision	0.839
Recall	0.929
ROC-AUC	0.961

The confusion matrix is `[[28, 5], [2, 26]]`, with true class labels as rows and predicted labels as columns. Thus, on this test split, 28 negative and 26 positive cases were classified correctly; five false positives and two false negatives were observed. This is one evaluation on a small dataset; I would not expect it to guarantee performance on a different population or in clinical use.

4. Experiment tracking and reproducibility

I track experiments under `heart_disease_assignment`. By default, the pipeline uses SQLite at `notebooks/mlflow.db` and local artifacts in `notebooks/mlruns/`; `MLFLOW_TRACKING_URI` can point to an alternate tracking store. Generated tracking files are excluded from Git.

The automated training run creates a parent model-selection run and 20 nested candidate runs. Candidate runs record model family, hyperparameters, and cross-validation mean and standard deviation for accuracy, precision, recall, and ROC-AUC. The parent run records the candidate count, fold count, dataset SHA-256, selected family and parameters, selected CV ROC-AUC, test metrics, confusion-matrix counts, classification report, and selected ONNX model artifact.

For the recorded run, MLflow reports Logistic Regression with `C=0.1`, 20 candidates, five folds, selected CV ROC-AUC 0.9113, test accuracy 0.8852, precision 0.8387, recall 0.9286, and test ROC-AUC 0.9610. I use a parent selection run and nested candidate runs to keep the comparison inspectable. The parent run includes the dataset digest, test results, classification report, and ONNX model; each candidate run includes its parameters and cross-validation metrics.

To reproduce the experiment, I install the project requirements and run the notebooks in sequence or execute `python -m src.pipeline`. The fixed seeds make the split and folds repeatable with compatible library versions, though they do not guarantee identical results across every software environment.

5. Model packaging and API

I convert the selected fitted preprocessing/classification pipeline to `artifacts/heart_disease_model.onnx`. `src.app` loads it with ONNX Runtime using the CPU execution provider. The Dockerfile installs the minimal serving requirements, copies the application and ONNX artifact, and starts Uvicorn with the FastAPI application.

I exposed two API routes:

- `GET /health` — returns `{"status":"ok"}` when the service is available;
- `POST /predict` — accepts the 13 named feature values as JSON and returns

the predicted class, label, predicted-class confidence, and positive-class probability.

The middleware writes method, path, HTTP status, and duration as structured JSON. It does not include request bodies or query strings in the logs.

The prediction is a model output, not a medical diagnosis. The service does not implement user authentication, so I use only synthetic sample data for demonstrations.

The request schema is a Pydantic model containing exactly the 13 feature fields. A successful response includes a binary integer `prediction`, a human-readable `label`, `confidence` for the predicted class, and `disease_probability` for the positive class. These values have different meanings: the positive-class probability is returned regardless of the predicted label, while confidence is the probability associated with the selected class. The ONNX runtime checks that the model emits a `(1, 2)` probability array; an unexpected output shape is treated as an explicit runtime error rather than producing a misleading response.

The service's lifespan loads the ONNX model once when the FastAPI application starts. `src.onnx_utils` converts the complete sklearn preprocessing/classifier pipeline; the ONNX tests compare predictions against the original sklearn estimators for Logistic Regression, Random Forest, and XGBoost. This keeps inference independent of loading the training notebook or the MLflow database in the serving container.

5.1 Running the project

I install the training and development requirements, then run the automated model-selection pipeline:

```
python -m pip install -r requirements.txt
python -m src.pipeline
python -m pytest -q
ruff check src tests
```

The pipeline downloads the dataset if it is missing, tracks the candidate runs, selects the model, and exports the ONNX artifact. To build and run the API container after the model artifact has been generated:

```
docker build -t heart-disease-api .
docker run --rm -p 8000:8000 heart-disease-api
```

I tested `GET /health` and `POST /predict` locally in CI and against the AWS deployment. The deployed sample request used synthetic values, not a real patient record.

6. CI/CD and automated testing

`.github/workflows/ci.yml` runs on pushes and pull requests targeting `main`. It installs `requirements.txt`, runs Ruff, executes Pytest, trains and tracks the candidate models, builds the Docker image, starts the API container, and smoke-tests `/health` and `/predict`. Training logs, MLflow files, the ONNX model, and the sample prediction response are uploaded as workflow artifacts even when a prior step fails.

My tests cover data/schema handling, missing-marker conversion, binary target mapping, candidate-grid composition, preprocessing and imputation, ONNX prediction parity, API responses, health checks, and structured request logging.

On successful CI for a push to `main`, the separate AWS deployment workflow assumes its AWS role using GitHub OIDC, builds and pushes the serving image to ECR, applies Terraform, waits for ECS capacity, and

smoke-tests the deployed health and prediction URLs. It can also be manually dispatched for deploy or destroy operations.

7. Deployment architecture

I used API Gateway as the public entry point and kept the application tasks and load balancer private. The deployment workflow provisions one ECS service on EC2 capacity. Its API container sends structured request logs to CloudWatch.

Architecture at a glance

Stage	Components and flow
Source and CI	GitHub repository ? GitHub Actions CI (lint, tests, training, Docker smoke test)
Delivery	Successful CI ? GitHub Actions with OIDC ? ECR and Terraform
Request path	Public API Gateway + WAF ? VPC Link ? internal ALB ? private ECS task on EC2
Application	FastAPI + ONNX Runtime serves /health and /predict
Monitoring	ECS structured logs ? CloudWatch Logs ? AWS Managed Grafana

The Terraform configuration provisions private subnets and VPC endpoints for ECS, ECR, CloudWatch Logs, and S3 instead of a NAT or internet gateway. The CloudWatch log group has a configured 14-day retention period. Terraform state is configured for an encrypted S3 backend with native S3 lockfiles; the GitHub OIDC bootstrap resources are managed separately.

The infrastructure is defined as code in `deploy/aws/main.tf` and related Terraform files. It creates an ECR repository, VPC, private subnets, VPC endpoints, API Gateway/WAF, VPC Link, internal Application Load Balancer, ECS cluster and service, EC2 Auto Scaling capacity, IAM roles, and CloudWatch logging. The configured ECS task uses bridge networking and maps API port 8000 to its EC2 host. Its security group allows this port from the internal ALB; administrative and monitoring ports are not opened by this application stack.

The public API entry point is the API Gateway HTTPS URL. API Gateway forwards through the VPC Link to the internal ALB and then to the ECS task. The AWS deployment workflow first creates and pushes the container image to ECR and provisions capacity, then applies the complete infrastructure and verifies the health and prediction endpoints. Its OIDC role avoids storing long-lived AWS access keys in GitHub. The AWS stack and supporting state bucket have ongoing costs while resources are provisioned; use the documented destroy workflow when the application environment is no longer needed, while retaining the separate state/bootstrap resources as described by the project.

Terraform provisions AWS Managed Grafana with a CloudWatch data source and an IAM role scoped to the API's CloudWatch log group. After Terraform creates the workspace, the deployment workflow uses a short-lived Grafana API key to upsert the data source, test Logs Insights access, and import the dashboard. The workflow deletes that API key when publishing ends. Workspace access is granted to IAM Identity Center users or groups configured for the deployment.

7.1 Deployment evidence

Deployed API base URL: https://getgd5jtgh.execute-api.ap-south-1.amazonaws.com/prod

Health check: GET https://getgd5jtgh.execute-api.ap-south-1.amazonaws.com/prod/health returned `{"status":"ok"}` on 18 October 2026.

Prediction check: POST https://getgd5jtgh.execute-api.ap-south-1.amazonaws.com/prod/predict returned `prediction=1`, `label="High risk"`, and confidence / positive-class probability `0.5896` for the documented synthetic sample input. This is an API smoke test, not a clinical result.

API/log group region: `ap-south-1`.

Managed Grafana region: `ap-southeast-1`, because AWS Managed Grafana is not available in `ap-south-1`.

Grafana dashboard: Open the deployed dashboard.

8. Monitoring and operational demonstration

The Managed Grafana dashboard queries the API's structured JSON CloudWatch events. Each event includes the HTTP method, path, status code, and elapsed time; request bodies and query strings are omitted. The dashboard shows request counts, HTTP errors, p95 latency, and recent requests. The deployment workflow checks Logs Insights access before importing the dashboard.

During deployment testing, I found that Uvicorn access logs were reaching CloudWatch but the structured request events were not. I configured the application logger to emit the JSON events explicitly, then adjusted the dashboard filters to match the events. I also changed the error panel to show zero when requests are present but none failed. I verified live structured events in CloudWatch and confirmed all four Grafana panel queries returned successfully with data frames. Most current events are load-balancer health checks; requests to `/predict` add prediction traffic to the dashboard.

The dashboard reports API activity; it does not measure model quality or prediction drift. Those would require separate, privacy-conscious data collection. CloudWatch logs and all four Grafana panel queries have been verified against the deployed service. I still need to capture the populated dashboard and API responses for the submission video. If I include a controlled invalid request, I will distinguish that test from normal traffic.

9. Limitations and future work

- The dataset has 303 records, and evaluation uses one stratified train/test split. Broader validation, temporal/external validation, and confidence intervals are outside the current project.
- I treated `ca` as a numeric count and `thal` as categorical. This is a modeling choice, not a claim that the feature encodings are the only valid interpretation.
- The classifier's probabilities have not been established as calibrated clinical probabilities. No clinical use is intended.
- The API is not authenticated. A real production service would require an appropriate authentication/authorization design, rate limiting, model governance, and stronger operational controls.

- The current Terraform ECS capacity is configured for one instance and a

single API task. Availability, autoscaling, and production capacity planning require further work.

- CloudWatch log-based monitoring is not a replacement for metrics when

numerical service or infrastructure metrics are required. This dashboard is intentionally based on structured API request logs.

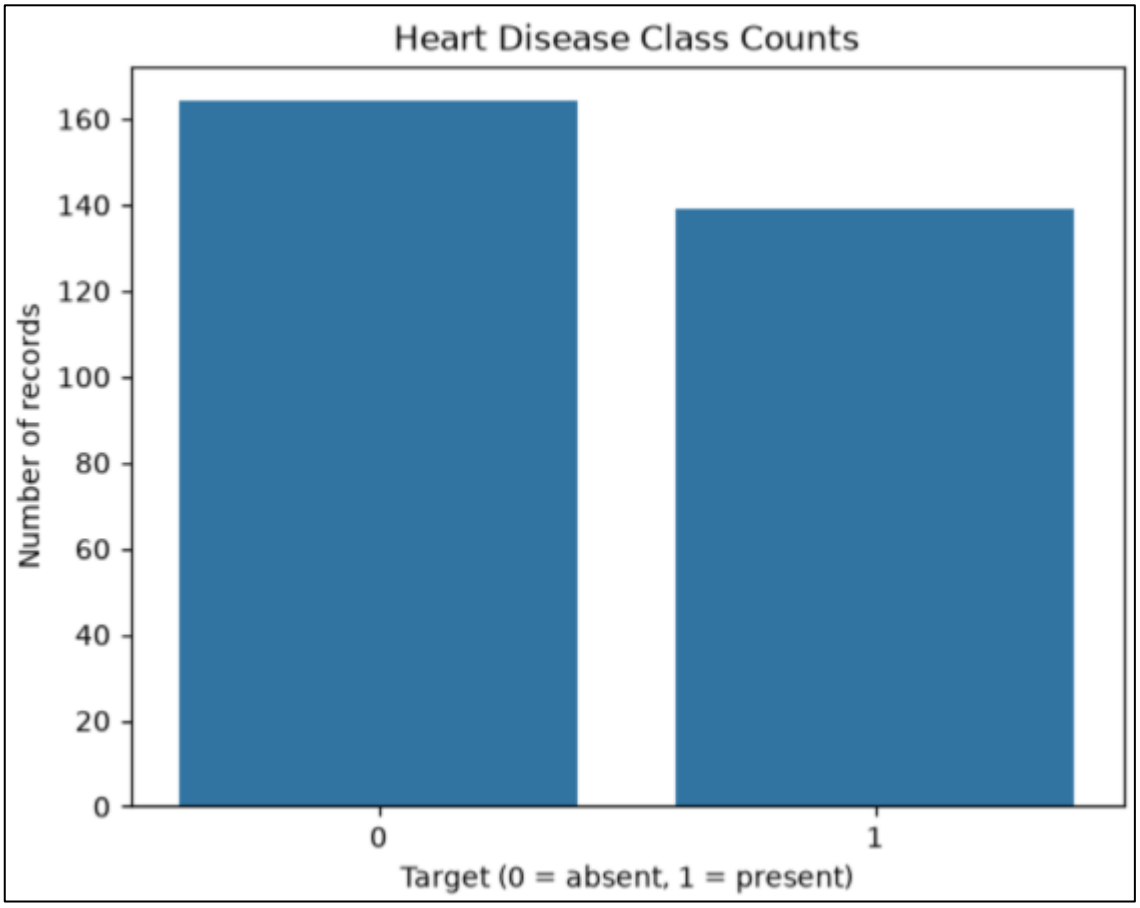
If I extend the project, I would start with external validation, operating- point analysis, alerting, authentication, and more scalable task capacity. Metrics beyond request logs would need a separate design for the private AWS service.

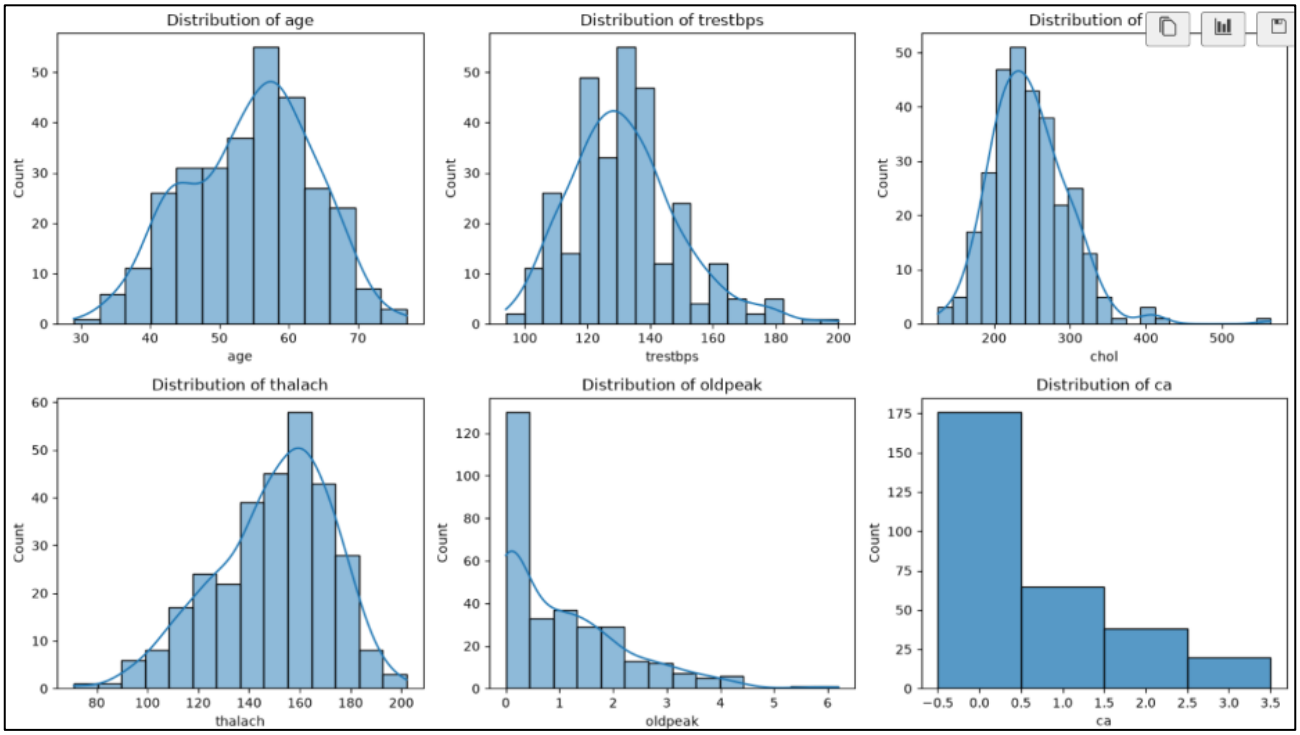
10. Conclusion

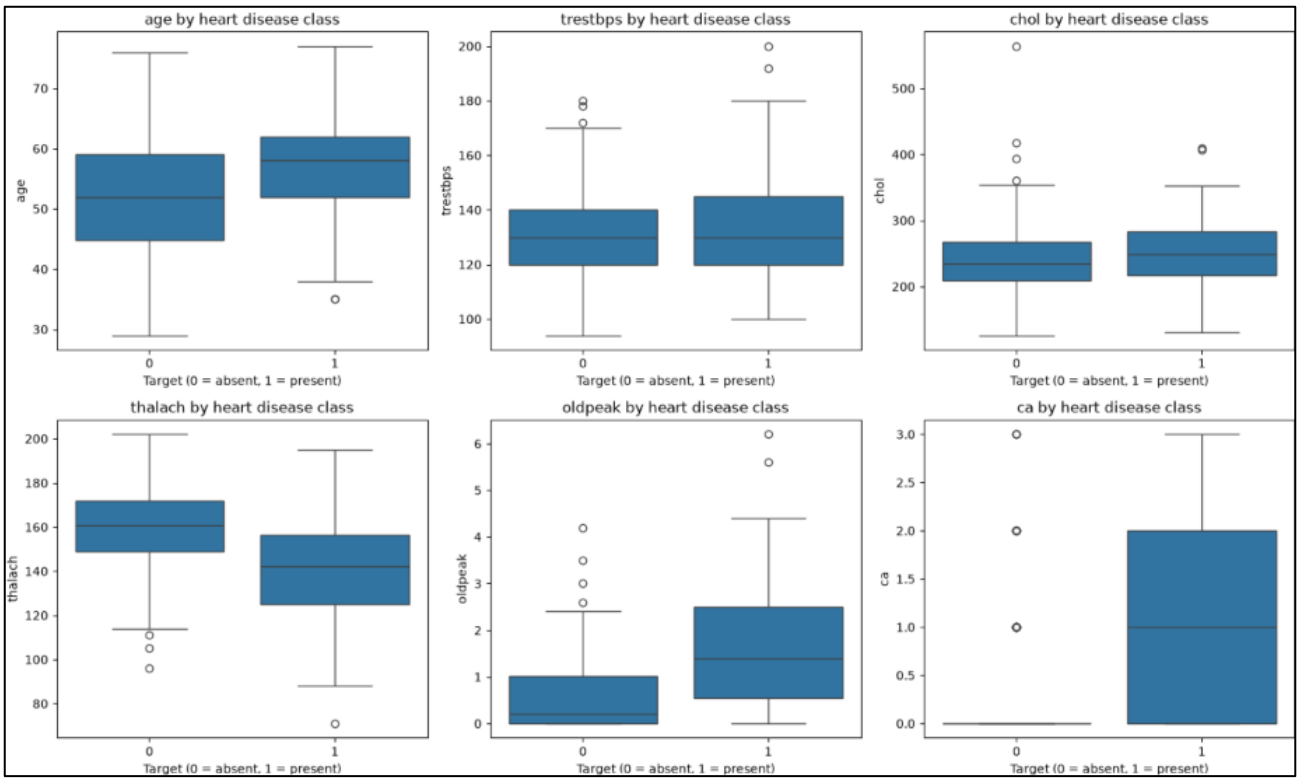
In this project I connected model development to an operational API: I cleaned and explored the dataset, compared models with cross-validation, tracked runs in MLflow, exported the selected pipeline to ONNX, and deployed a Dockerized FastAPI service to AWS. The recorded Logistic Regression model achieved 0.911 mean CV ROC-AUC and 0.961 held-out test ROC-AUC, with test recall of 0.929. I also verified the public API, CI and deployment workflows, CloudWatch request events, and Grafana panel queries. The evaluation remains limited by the dataset size and single test split, so I do not treat it as evidence of clinical

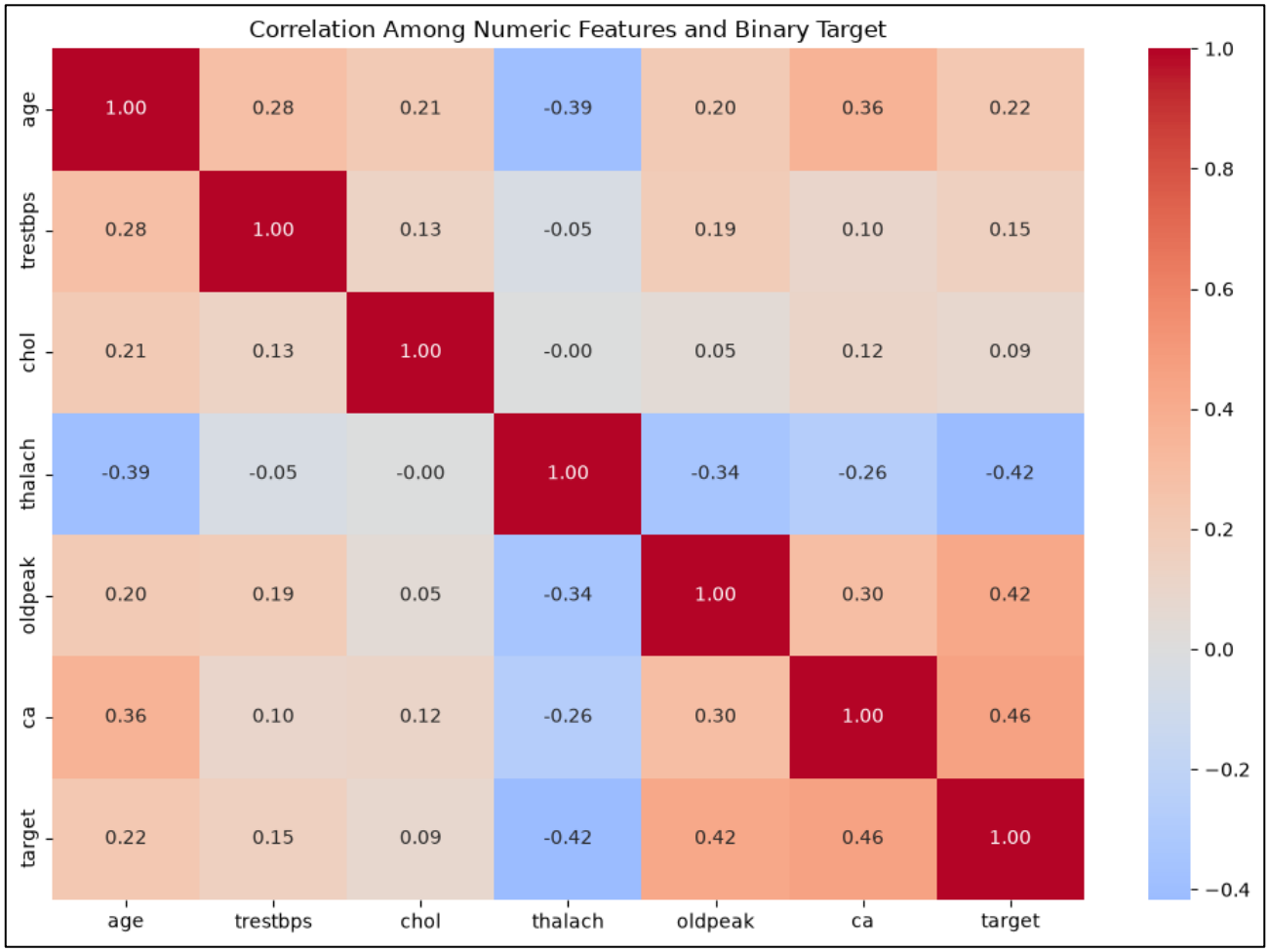
Appendix A. Required submission evidence

1. EDA plots and data-quality summary.




Distribution of age
Count
age
Distribution of trestbps
Count
trestbps
Distribution of
Count
chol
Distribution of thalach
Count
thalach
Distribution of oldpeak
Count
oldpeak
Distribution of ca
Count
ca


age by heart disease class
age
Target (0 = absent, 1 = present)
trestbps by heart disease class
trestbps
Target (0 = absent, 1 = present)
chol by heart disease class
chol
Target (0 = absent, 1 = present)
thalach by heart disease class
thalach
Target (0 = absent, 1 = present)
oldpeak by heart disease class
oldpeak
Target (0 = absent, 1 = present)
ca by heart disease class
ca
Target (0 = absent, 1 = present)

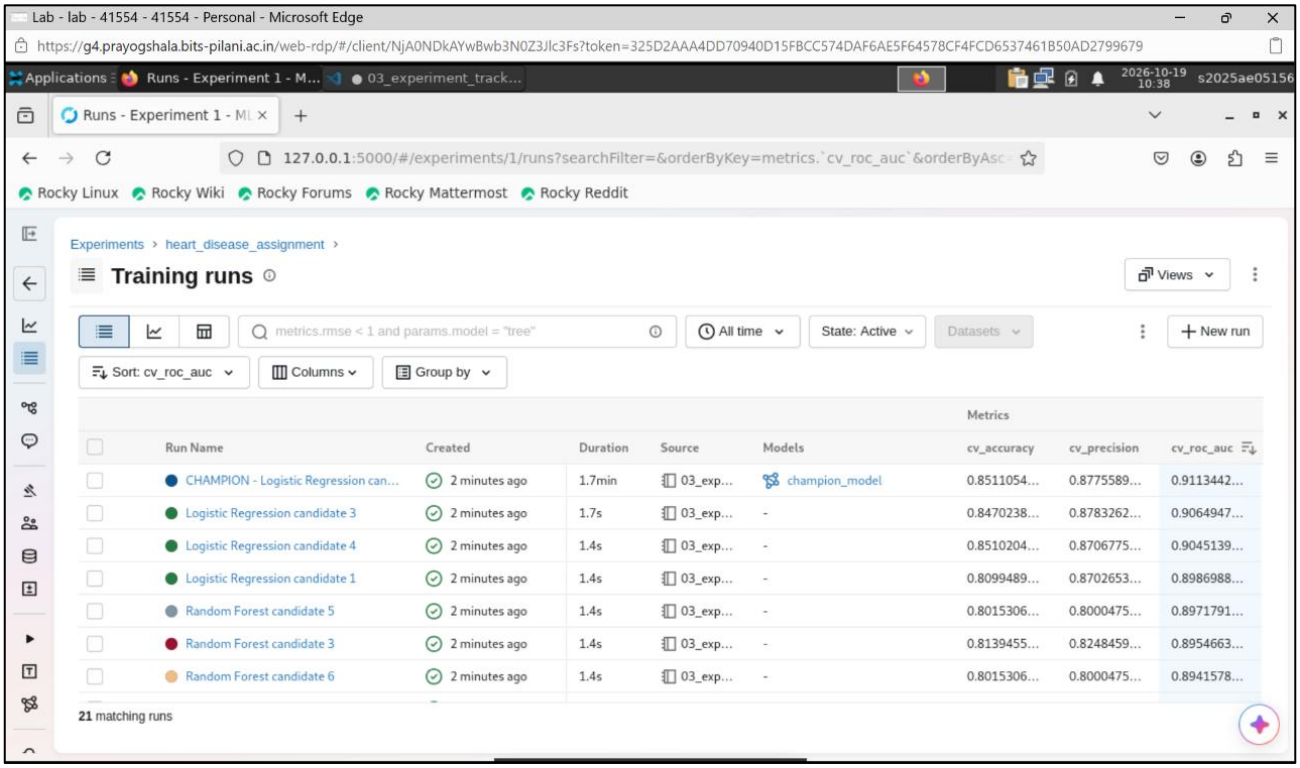


2. Model-comparison results and selected model details.

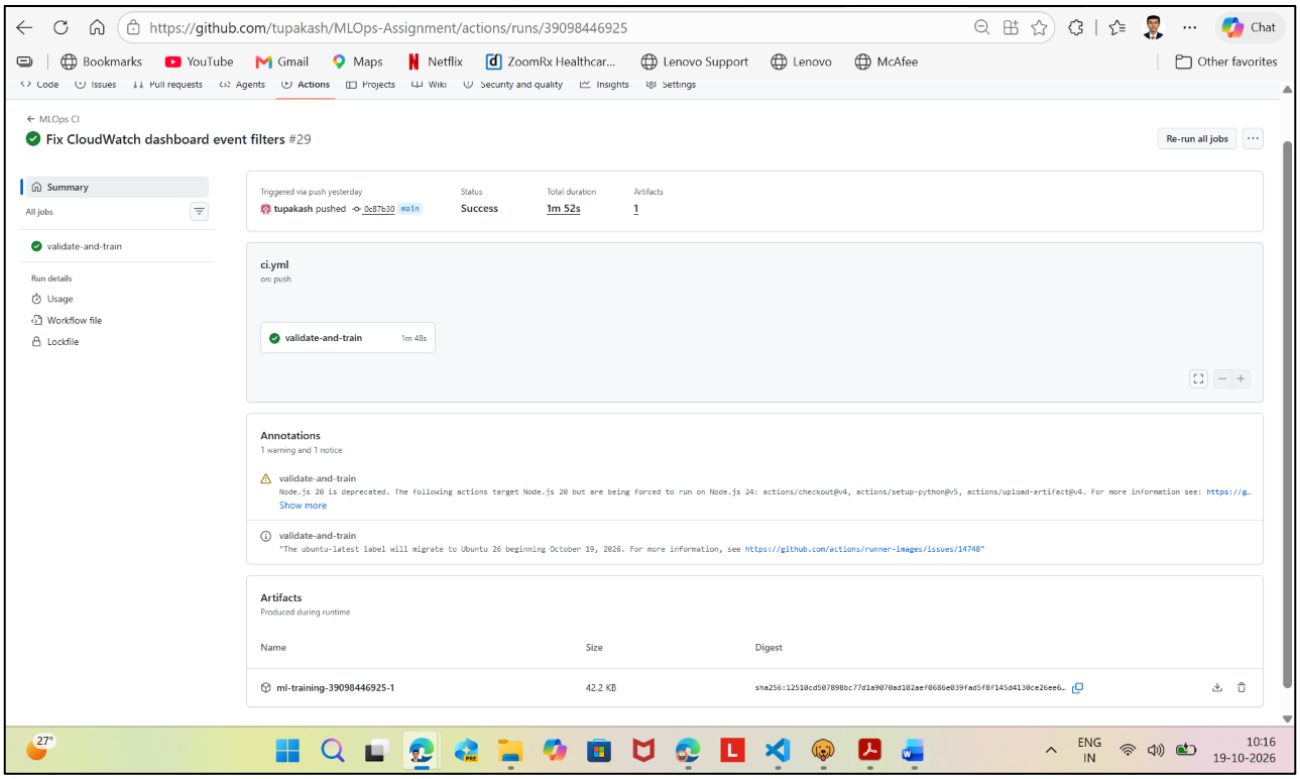
	model	cv_accuracy	cv_precision	cv_recall	cv_roc_auc	best_parameters
0	Logistic Regression	0.851	0.878	0.792	0.911	{'classifier__C': 0.1}
1	Random Forest	0.802	0.800	0.765	0.897	{'classifier__max_depth': 8, 'classifier__min_...
2	XGBoost	0.822	0.828	0.783	0.885	{'classifier__learning_rate': 0.03, 'classifie...

```
Selected model: Logistic Regression
Best parameters: {'classifier__C': 0.1}
```

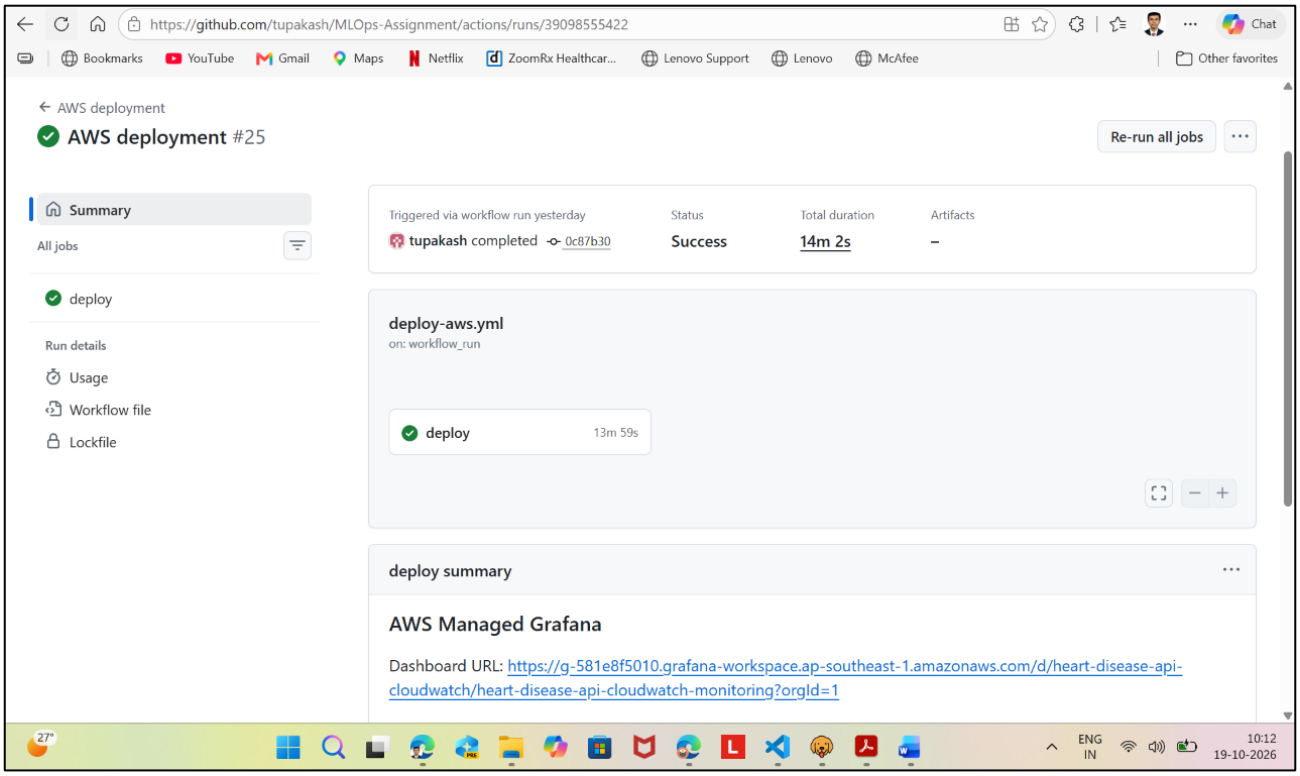
3. MLflow experiment page showing the parent run, candidate runs, and logged artifacts.

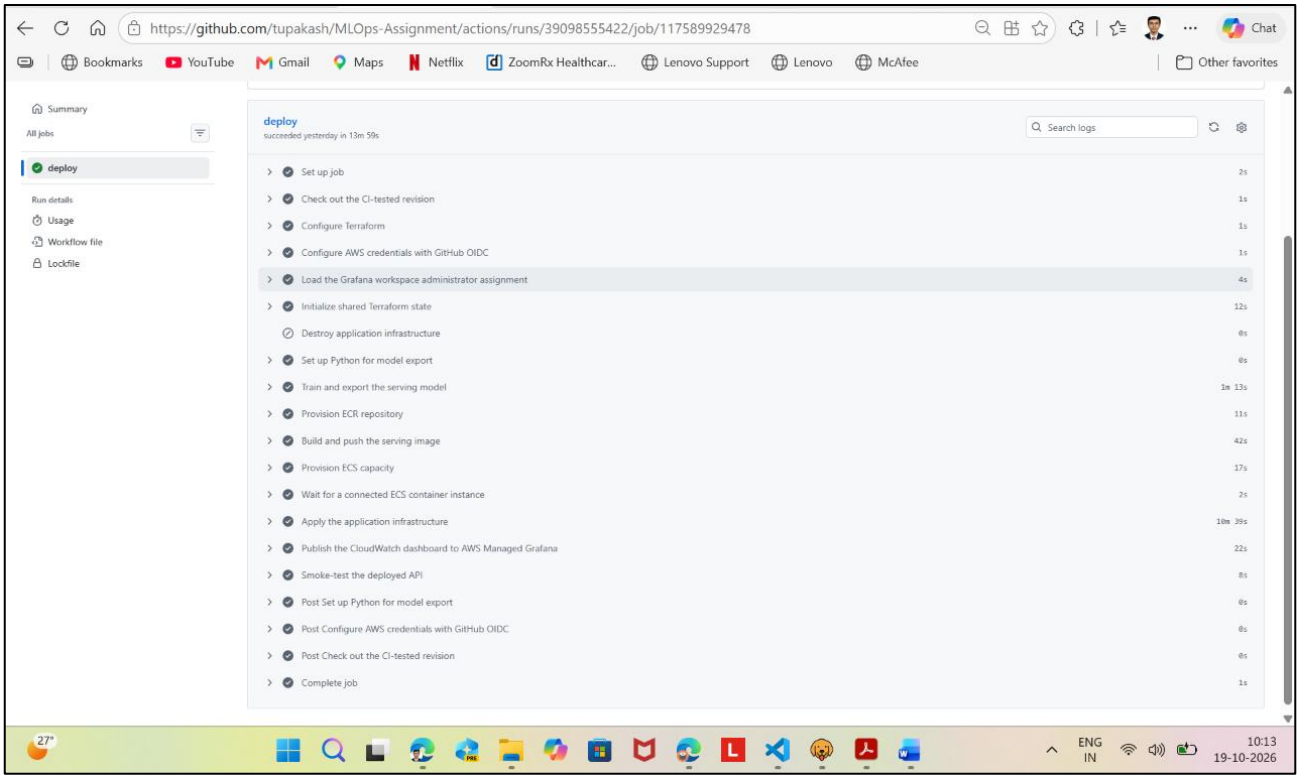


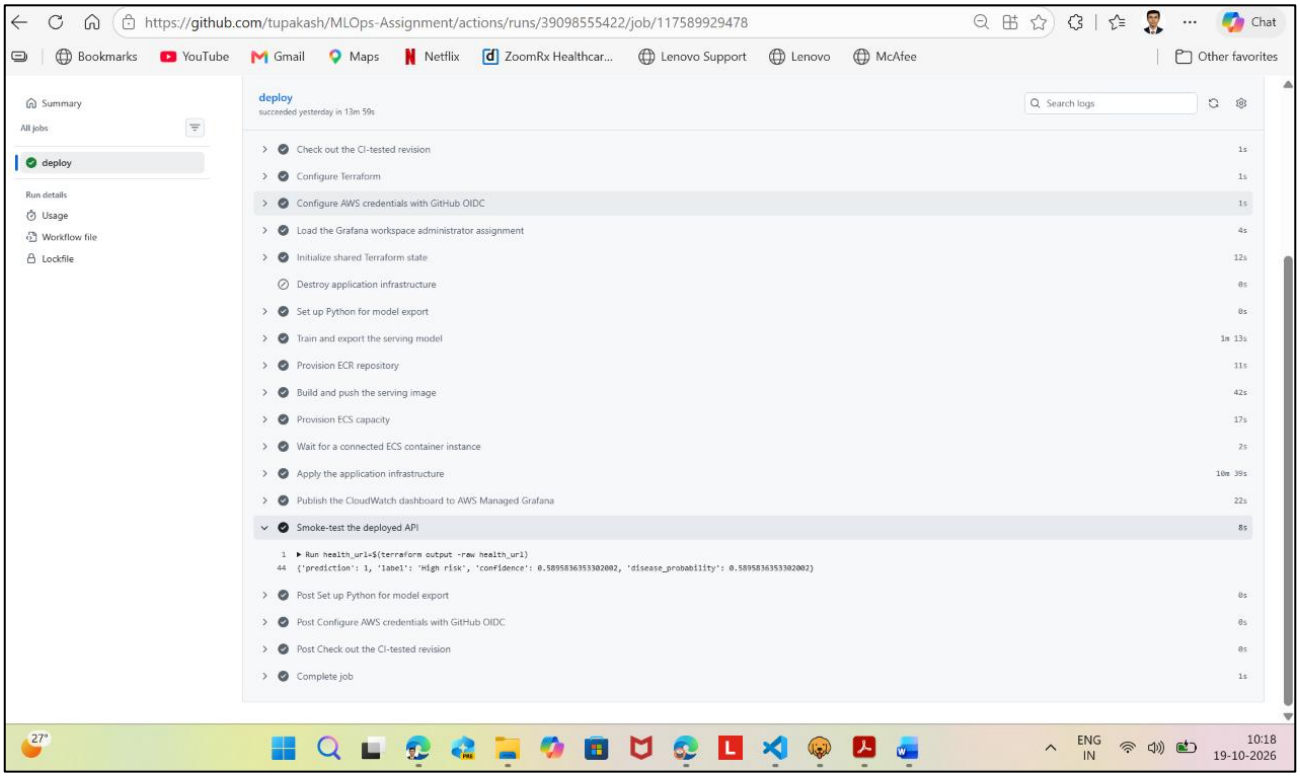
4. Latest CI Workflow.



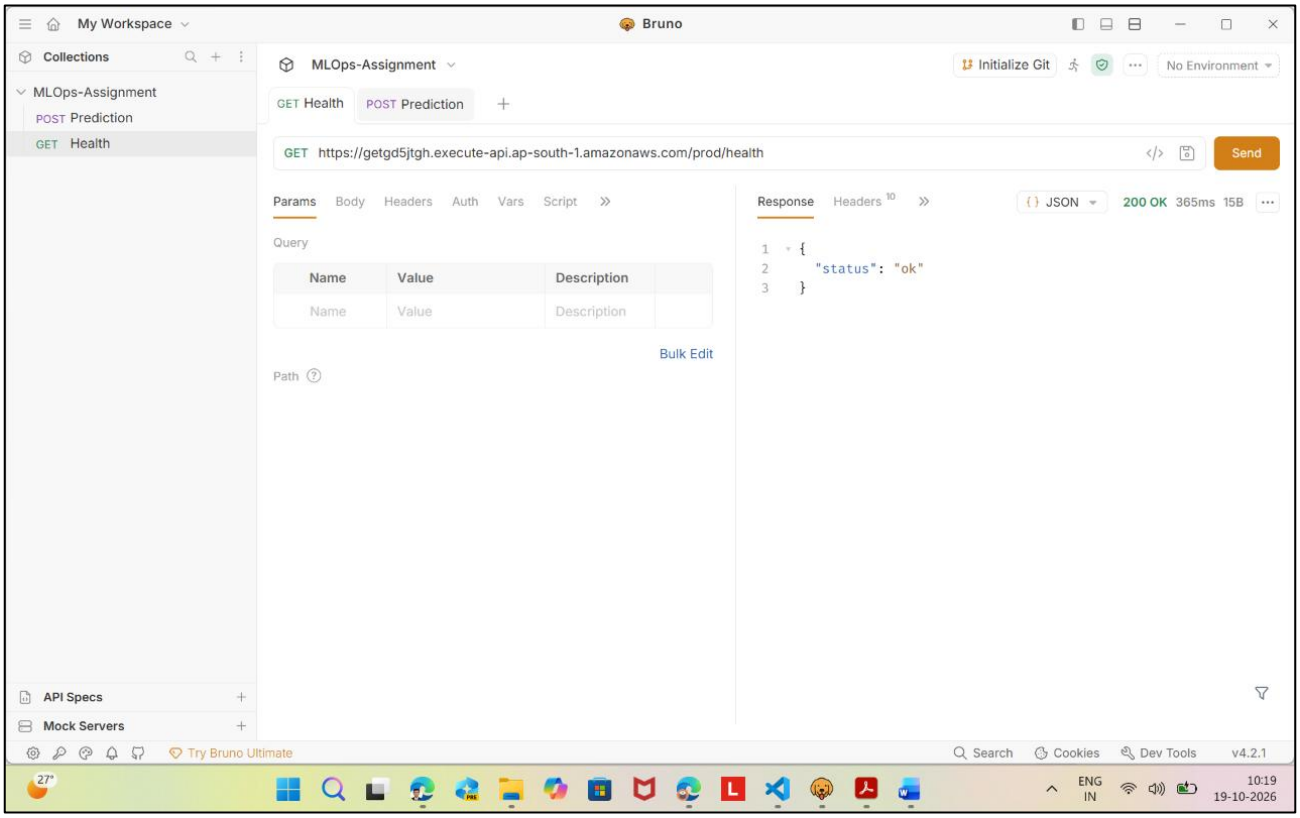
5. Successful AWS deployment workflow.

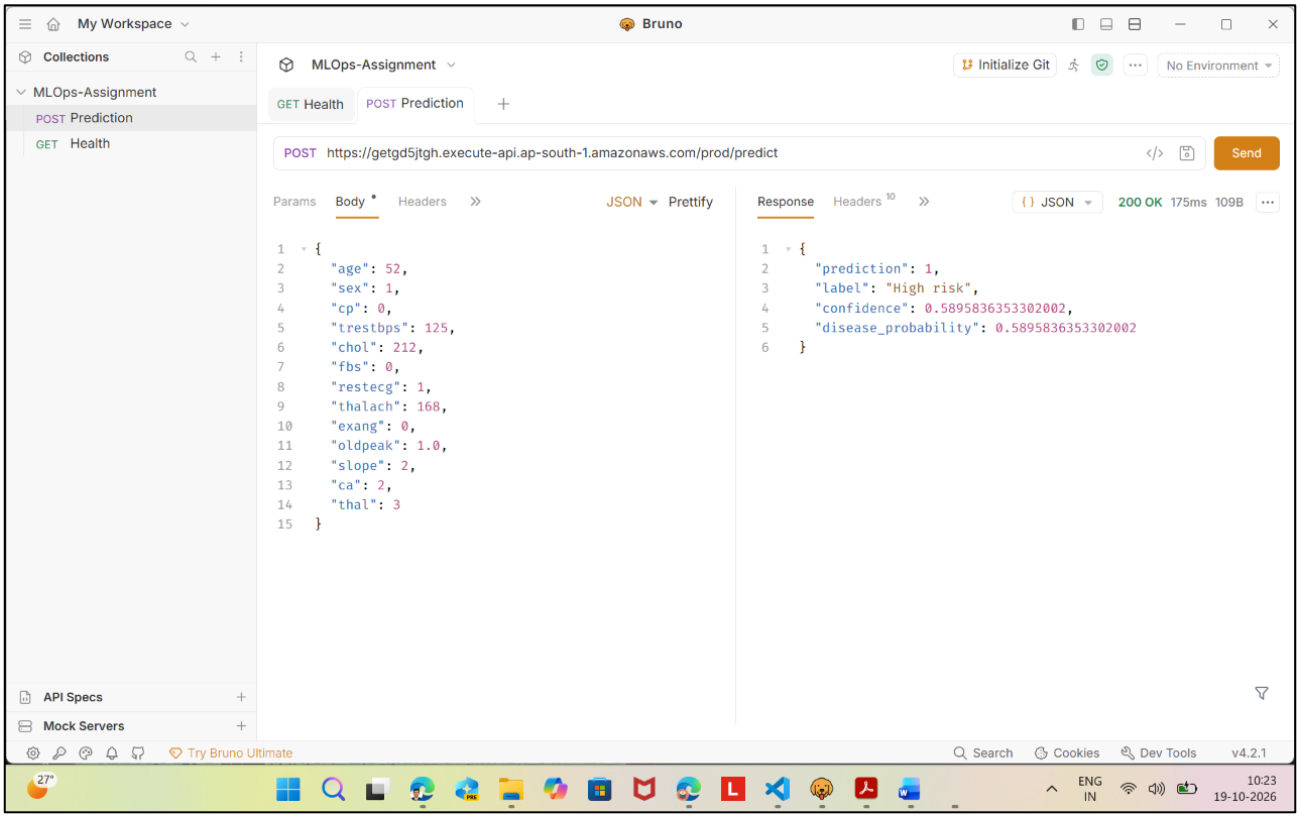




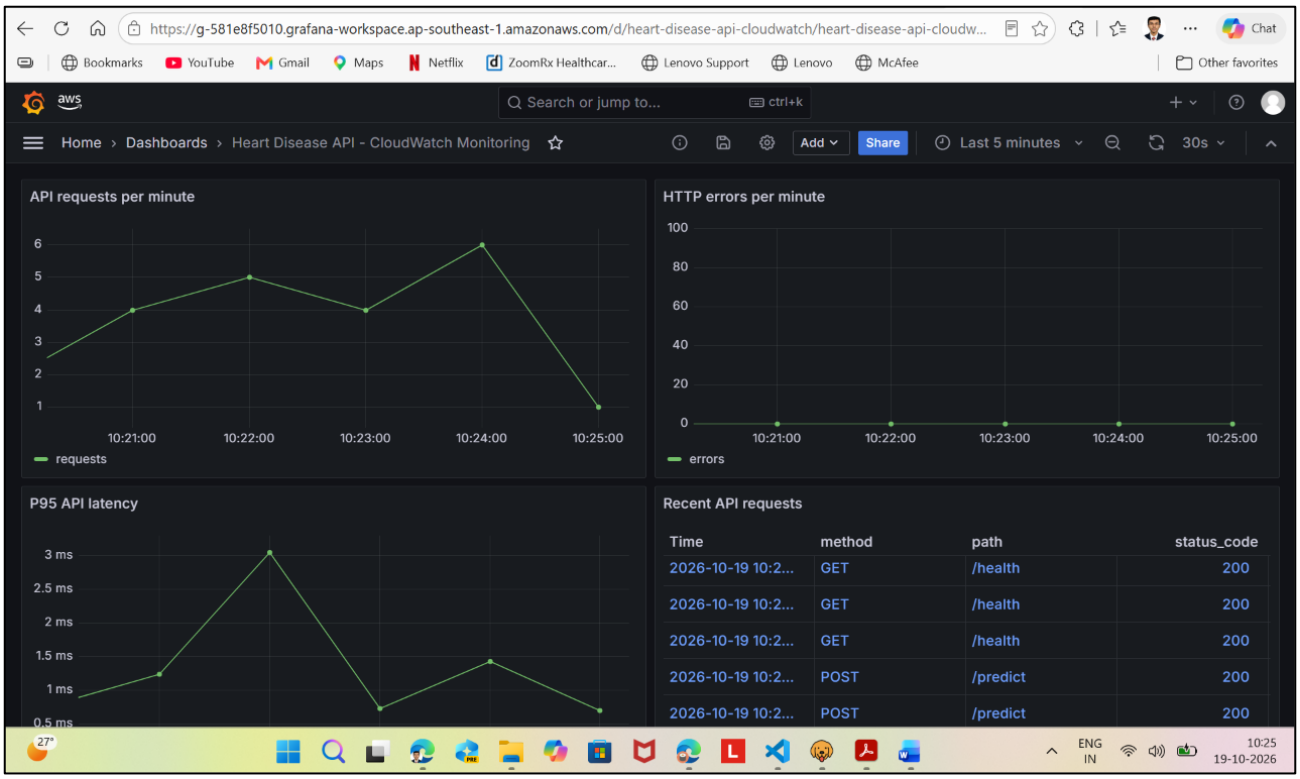


6. Bruno screenshots for health and prediction calls.

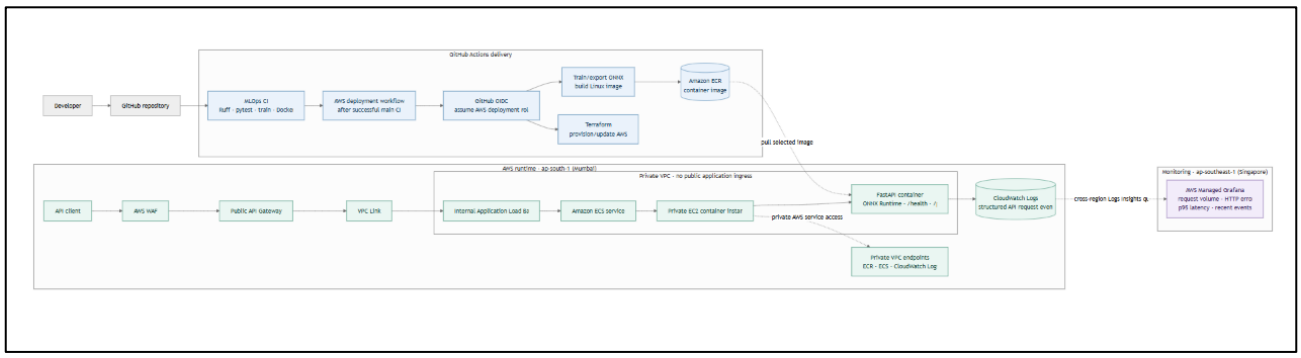




7. Grafana Dashboard



8. Deployment architecture


GitHub Actions delivery
Developer
GitHub repository
MLOps CI
Ruff · pytest · train · Docker
AWS deployment workflow
after successful main CI
GitHub OIDC
assume AWS deployment role
Train/export ONNX
build Linux image
Amazon ECR
container image
Terraform
provision/update AWS
pull selected image
AWS runtime · ap-south-1 (Mumbai)
Private VPC · no public application ingress
API client
AWS WAF
Public API Gateway
VPC Link
Internal Application Load Bal
Amazon ECS service
Private EC2 container instar
private AWS service access
FastAPI container
ONNX Runtime · /health · /i
Private VPC endpoints
ECR · ECS · CloudWatch Log
Cloudwatch Logs
structured API request even
cross-region Logs Insights qu
Monitoring · ap-southeast-1 (Singapore)
AWS Managed Grafana
request volume · HTTP error
p95 latency · recent events